



Atelier Complet Full Stack

2-BackEnd 2

Khalid Nafil
ENSIAS

Spring Rest

1. A ce stade, nous allons créer un webservice RESTful. Nous commençons par créer la classe VoitureController dans le package 'org.cours.web'.
2. Cette classe va être annotée par @RestController pour indiquer qu'elle va jouer le rôle du contrôleur du webservice RESTfull.
3. Nous allons ajouter une méthode Iterable qui va être annotée par @RequestMapping qui définit le 'endpoint' ("/voitures") avec lequel la méthode va être mappée. Cette annotation permet de traiter toutes les méthodes HTTP (GET, POST, PUT, PATCH, DELETE, etc), il suffit de faire @RequestMapping("/voitures", method=GET) pour définir le endpoint 'voitures' qui va être manipulé par la méthode GET. Nous allons nous contenter maintenant de la méthode GET, qui est la méthode utilisée par défaut. On va injecter ensuite la classe **voitureRepo** pour pouvoir utiliser sa méthode findAll() qui va nous retourner toutes les voitures de la Repository. Le code est comme suit :

@RestController

```

public class VoitureController {

    @Autowired

    private VoitureRepo voitureRepo;

    @RequestMapping("/voitures")

    public Iterable<Voiture> getVoitures(){

        return voitureRepo.findAll();

    }

}

```

4. Allez après sur Postman pour faire les tests. Choisir la méthode 'GET' et 'localhost:8080/voitures'. Appuyez sur 'Send' et vous allez remarquer qu'il y a un effet de boucle dû à la sérialisation répétée entre Voiture et Proprietaire. Pour stopper cet effet, nous allons ajouter l'annotation @JsonIgnore avant l'attribut de jointure dans les deux classes Voiture et Proprietaire, comme suit :

//Proprietaire.java

```
@OneToMany(cascade = CascadeType.ALL, mappedBy = "proprietaire")
```

```
@JsonIgnore
```

```
private List<Voiture> voitures;
```

//-----

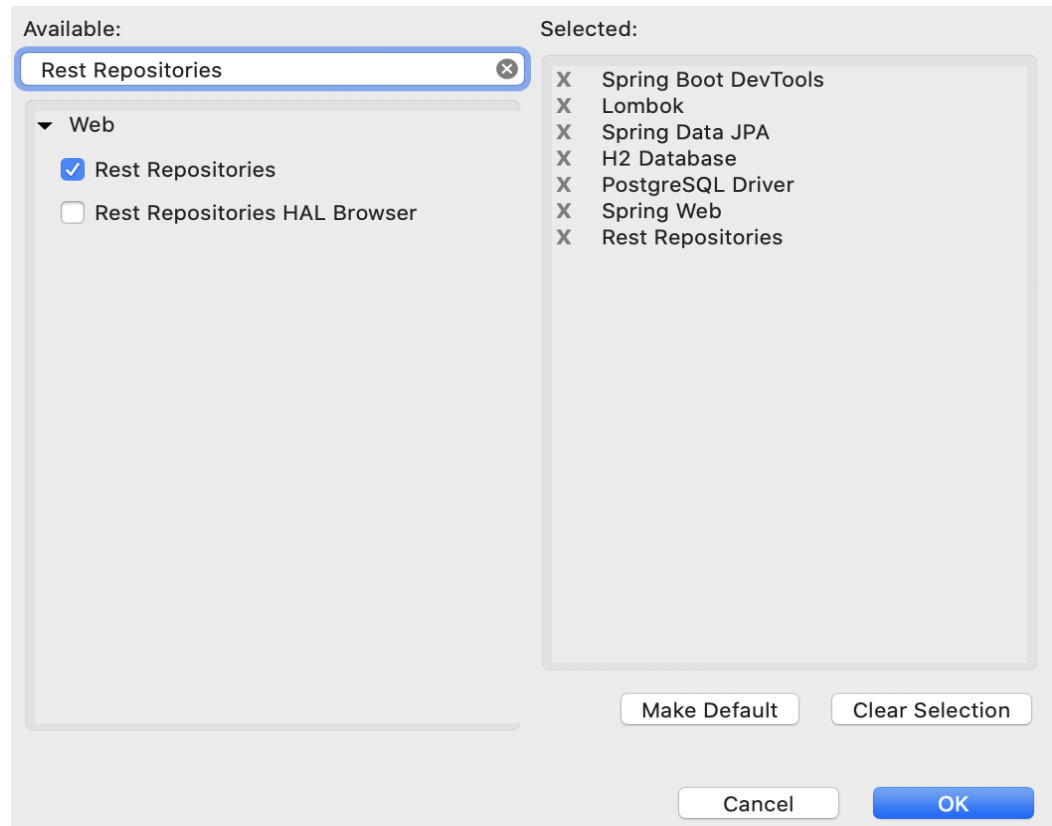
//Voiture.java

```
@JoinColumn(name = "proprietaire")
```

```
@JsonIgnore
```

```
private Proprietaire proprietaire;
```

5. Maintenant vous pouvez ré-exécuter et voir le résultat qui affiche la liste des voitures sur 'localhost:8080/voitures', avec POSTMAN.
6. Voilà !!!
7. Maintenant nous allons intégrer la dépendance Rest Repositories dans notre projet. Faites le (bouton droit sur le nom du projet, ensuite 'Spring', ensuite 'Edit Starters' et choisir 'Rest Repositories' puis validez par Ok :



8. Vous allez après ajouter au fichier 'application.properties' la ligne suivante : `spring.data.rest.base-path=/api` , cela va permettre de définir un endpoint nommé 'api' pour gérer toutes les méthodes HTTP (GET, POST, PUT, etc).
9. Redémarrez le serveur et tapez l'adresse <http://localhost:8080/api>
10. Vous allez voir l'affichage des données JSON dans le format HAL (HyperText Application Language). Le format HAL fournit un ensemble de conventions pour exprimer les hyperliens sous JSON et faire en sorte que votre webservice RESTfull soit facile à utiliser par les développeurs Frontend.
11. Si vous choisissez l'url <http://localhost:8080/api/voitures> vous allez pouvoir récupérer la liste de toutes les voitures

GET http://localhost:8080/api/voitures Send

Params Authorization Headers (7) Body Pre-request Script Tests Settings

Query Params

KEY	VALUE	DESCRIPTION	...
Key	Value	Description	

Body Cookies Headers (8) Test Results Status: 200 OK Time: 131 ms Size: 1.71 KB Save F

Pretty Raw Preview Visualize JSON ↺

```

1 {
2   "_embedded": {
3     "voitures": [
4       {
5         "marque": "Ford",
6         "modele": "Fiesta",
7         "couleur": "Grise",
8         "prix": 90000,
9         "_links": {
10          "self": {
11            "href": "http://localhost:8080/api/voitures/3"
12          },
13          "voiture": {
14            "href": "http://localhost:8080/api/voitures/3"
15          },
16          "proprietaire": {
17            "href": "http://localhost:8080/api/voitures/3/proprietaire"
18          }
19        }
20      }
21    ]
22  }

```

12. Le service Spring Data Rest fournit toutes les opérations CRUD. GET: Read, POST: Create, PUT/PATCH: Update, DELETE: Delete.
13. Pour supprimer la voiture par exemple dont l'id est 3, on tape l'url suivante : <http://localhost:4545/api/voitures/3> via l'outil POSTMAN. Amusez-vous à tester toutes les autres méthodes : GET, POST, PATCH et PUT.
14. Pour inclure maintenant ces requêtes dans notre service, nous allons ajouter l'annotation `@RepositoryRestResource` à la classe repository. Les paramètres de la requête sont annotés par l'annotation `@Param`. Nous allons appliquer cela à la classe VoitureRepo :
15. `@RepositoryRestResource`

```

public interface VoitureRepo extends CrudRepository<Voiture, Long>{

    //Lister Voitures par marque

    List<Voiture> findByModele(@Param("modele") String modele);

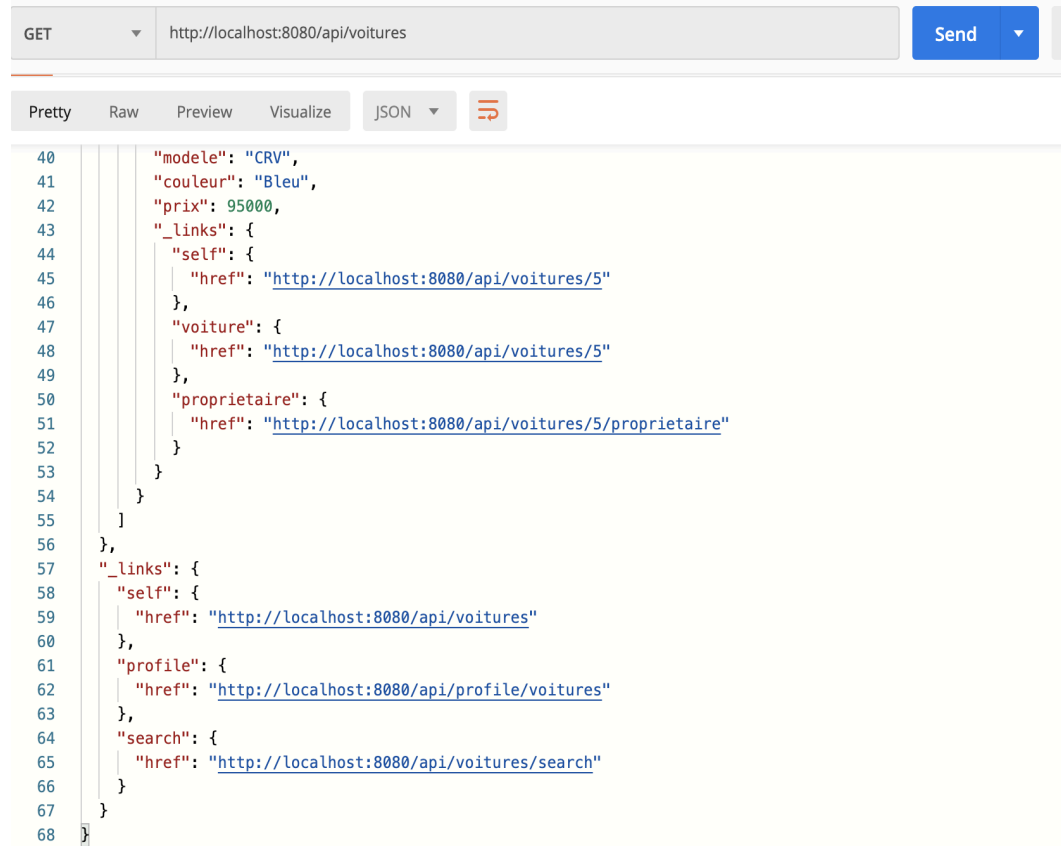
    //Lister Voitures par couleur

```

```
List<Voiture> findByCouleur(@Param("couleur") String couleur);

}
```

16. On exécute le projet avec GET pour l'api voitures, on va s'apercevoir qu'une nouvelle branche 'search' est ajoutée que vous pouvez explorer.



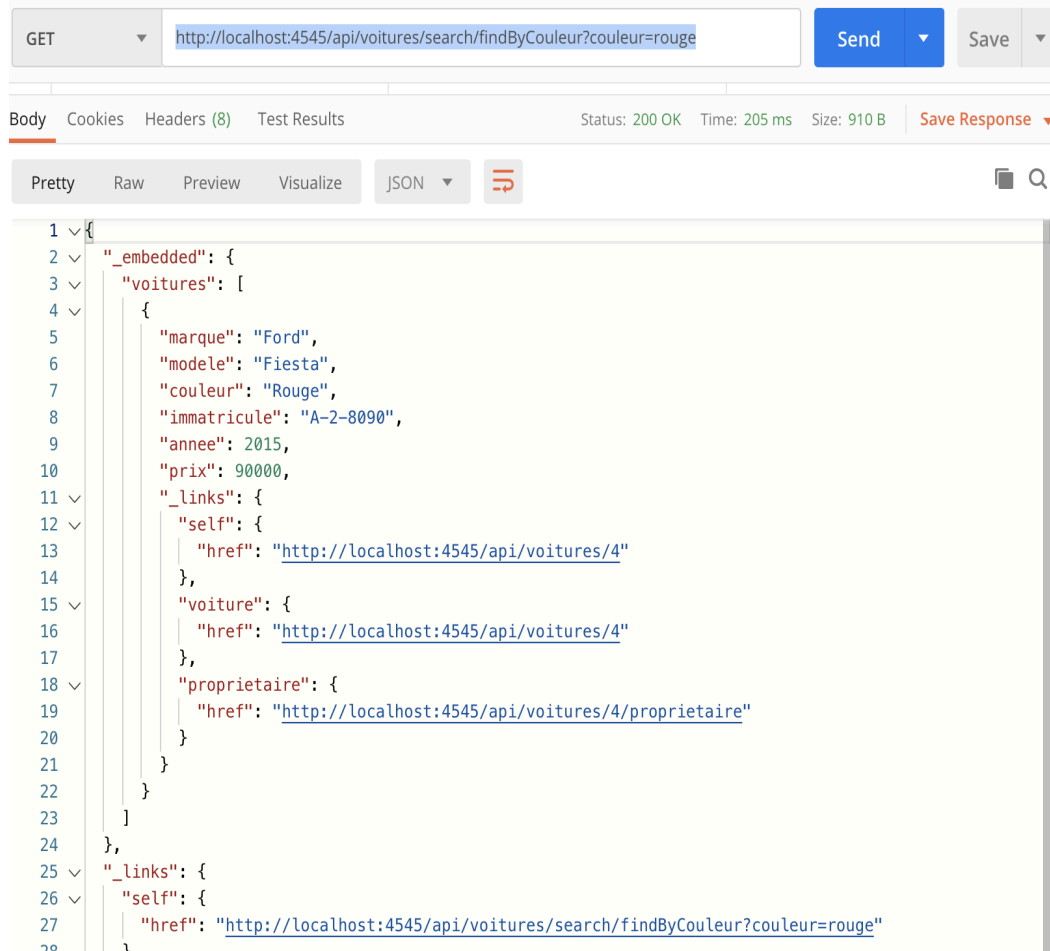
```
40      "modele": "CRV",
41      "couleur": "Bleu",
42      "prix": 95000,
43      "_links": {
44        "self": {
45          "href": "http://localhost:8080/api/voitures/5"
46        },
47        "voiture": {
48          "href": "http://localhost:8080/api/voitures/5"
49        },
50        "proprietaire": {
51          "href": "http://localhost:8080/api/voitures/5/proprietaire"
52        }
53      }
54    },
55  ],
56  "_links": {
57    "self": {
58      "href": "http://localhost:8080/api/voitures"
59    },
60    "profile": {
61      "href": "http://localhost:8080/api/profile/voitures"
62    },
63    "search": {
64      "href": "http://localhost:8080/api/voitures/search"
65    }
66  }
67 }
68 }
```

Cliquez sur cette branche et valider en appuyant sur le bouton 'Send' de Postman.

17. Veuillez sélectionner par exemple la branche des couleurs et s'assurer que dans l'url de Postman vous avez ça :

`http://localhost:4545/api/voitures/search/findByCouleur?couleur=Rouge`

18. Si vous validez vous allez obtenir la réponse suivante :



```
1 {
2   "_embedded": {
3     "voitures": [
4       {
5         "marque": "Ford",
6         "modele": "Fiesta",
7         "couleur": "Rouge",
8         "immatricule": "A-2-8090",
9         "annee": 2015,
10        "prix": 90000,
11        "_links": {
12          "self": {
13            "href": "http://localhost:4545/api/voitures/4"
14          },
15          "voiture": {
16            "href": "http://localhost:4545/api/voitures/4"
17          },
18          "proprietaire": {
19            "href": "http://localhost:4545/api/voitures/4/proprietaire"
20          }
21        }
22      }
23    ]
24  },
25  "_links": {
26    "self": {
27      "href": "http://localhost:4545/api/voitures/search/findByCouleur?couleur=rouge"
28    }
29  }
30 }
```

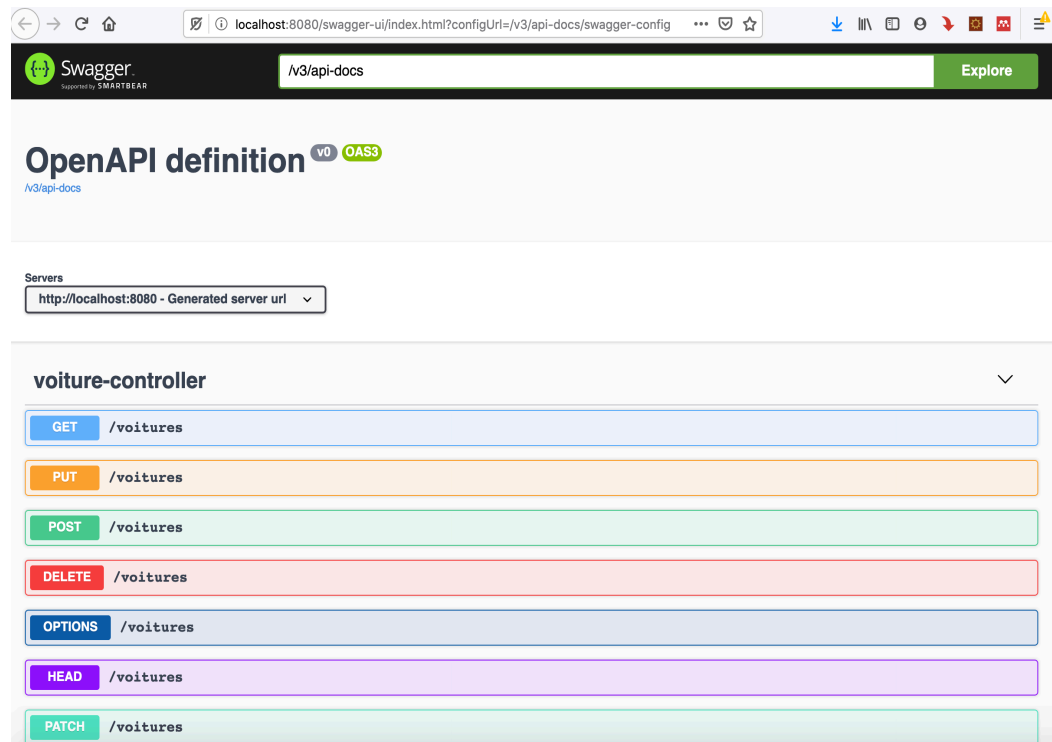
Springdoc-openapi

1. La librairie java **springdoc-openapi** aide à automatiser la génération de la documentation API utilisant les projets spring boot. Cette librairie fonctionne en examinant une application en mode exécution pour inférer les sémantiques de l'API en se basant sur les configurations de Spring, la structure des classes et les différentes annotations. Springdoc-openapi se veut une bonne alternative au célèbre [Swagger](#).
2. Springdoc-openapi génère automatiquement la documentation dans les formats APIs JSON/Yaml et HTML. Cette documentation peut éventuellement être complétée par l'usage des annotations de Swagger. Cette librairie supporte OpenAPI 3, Spring-boot (v1 et v2), JSR-303, Swagger-ui et OAuth 2.

3. Pour utiliser cette librairie dans notre projet, il suffit d'ajouter la dépendance suivante au niveau du fichier pom.xml :

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-ui</artifactId>
  <version>1.4.0</version>
</dependency>
```

4. Relancez votre projet et consultez les deux chemins : <http://localhost:8080/swagger-ui.html> qui va vous permettre de consulter votre documentation Rest sous une forme agréable bien sûr en demandant les services de swagger-ui.



Vous pouvez par exemple appuyer sur le bouton GET pour voir sa documentation et vous avez aussi la possibilité d'exécuter la méthode pour avoir le rendu suivant :

Request URL

<http://localhost:8080/voitures>

Server response

Code	Details
200	Response body <pre>[{ "id": 3, "marque": "Ford", "modele": "Fiesta", "couleur": "Gris", "prix": 90000 }, { "id": 4, "marque": "Toyota", "modele": "Corolla", "couleur": "Rouge", "prix": 80000 }, { "id": 5, "marque": "Honda", "modele": "CRV", "couleur": "Bleu", "prix": 95000 }]</pre> Download

L'autre chemin <http://localhost:8080/v3/api-docs> qui va lui vous permettre de générer la documentation dans sa forme basique.

localhost:8080/v3/api-docs

JSON Données brutes En-têtes

Enregistrer Copier Tout réduire Tout développer Filtre le JSON

```
openapi: "3.0.1"
info:
  title: "OpenAPI definition"
  version: "v0"
servers:
  0:
    url: "http://localhost:8080"
    description: "Generated server url"
paths:
  /voitures:
    get:
      tags:
        0: "voiture-controller"
      operationId: "getVoitures_3"
      responses:
        200:
          description: "OK"
          content:
            */*:
              schema: {}
    put:
      tags:
        0: "voiture-controller"
      operationId: "getVoitures_1"
      responses:
        200:
          description: "OK"
          content:
            */*:
              schema: {}
    delete:
      tags:
        0: "voiture-controller"
      operationId: "getVoitures_1"
      responses:
        200:
          description: "OK"
          content:
            */*:
              schema: {}
    options:
      tags:
        0: "voiture-controller"
      operationId: "getVoitures_1"
      responses:
        200:
          description: "OK"
          content:
            */*:
              schema: {}
    head:
      tags:
        0: "voiture-controller"
      operationId: "getVoitures_1"
      responses:
        200:
          description: "OK"
          content:
            */*:
              schema: {}
    patch:
      tags:
        0: "voiture-controller"
      operationId: "getVoitures_1"
      responses:
        200:
          description: "OK"
          content:
            */*:
              schema: {}
```


Spring Security

1. Spring Security fournit les services de sécurité pour les applications web à base de Java.
2. Commençons par ajouter la dépendance Spring Security à notre projet Spring boot.
3. Lorsque vous lancez votre application, un utilisateur va être ajouté avec le nom '**user**' et le password vous allez le voir sur la console, par exemple : **263ab9a7-c5da-40c2-9660-aecd09b1937b**
4. Si vous essayez maintenant de faire le GET, vous aurez un message d'erreur comme quoi vous n'êtes pas autorisé.
5. Pour pouvoir faire l'appel du GET, on va faire une authentification basique. Sur Postman, sélectionnez au niveau de l'onglet 'Authorization' le Type 'Basic Auth' et renseignez après votre 'user' et 'password'.

Le Test sous Spring Boot

1. Commencez par ajouter la dépendance suivante au niveau du pom.xml pour pouvoir faire le test sous H2 :
2. Veuillez ouvrir la classe générée par défaut par Spring Boot et qui va s'occuper de réaliser les tests : 'SpringbootReactPart1ApplicationTests' et la modifier de cette façon:

```
package org.cours;
import static org.assertj.core.api.Assertions.assertThat;
import org.cours.web.VoitureController;
import org.junit.jupiter.api.Test;
import org.junit.runner.RunWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.junit4.SpringRunner;
@RunWith(SpringRunner.class)
```

```

@SpringBootTest
//indique que c'est une classe de test régulier qui exécute les tests à base de Spring boot
class SpringbootReactPart1ApplicationTests {
    @Autowired
    VoitureController voitureController;
    @Test
    //indique que cette méthode peut être exécutée comme un cas de test
    void contextLoads() {
        assertThat(voitureController).isNotNull();
        //on teste si l'instance du contrôleur a été créé et injectée avec succès.
    }
}

```

3. Faites un bouton droit sur cette classe de test et choisissez Run As -> Junit Test. Si tout se passe bien, une barre verte va s'afficher, cela veut dire techniquement que le test passe.
4. Maintenant nous allons créer une classe de test pour les opérations CRUD de notre repository Voiture. Pour cela on crée la classe de test VoitureRepoTest comme suit :

```

package org.cours;
import static org.assertj.core.api.Assertions.assertThat;
import org.cours.modele.Voiture;
import org.cours.modele.VoitureRepo;
import org.junit.jupiter.api.Test;
import org.junit.runner.RunWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
import org.springframework.boot.test.autoconfigure.orm.jpa.TestEntityManager;
import org.springframework.test.context.junit4.SpringRunner;
@RunWith(SpringRunner.class)
@DataJpaTest
//Si le test concerne uniquement les composantes JPA
//Lorsque cette annotation est utilisée, H2, Hibernate et Spring Data sont configurés
//automatiquement pour le test.
public class VoitureRepoTest {
    @Autowired
    private TestEntityManager entityManager;
    //TestEntityManager est utilisée pour manipuler les entités persistantes
    @Autowired
    VoitureRepo voitureRepo;
    @Test

```

```

public void ajouterVoiture() {
    Voiture voiture = new Voiture ("MiolaCar","Uber","Blanche","M-2020", 2021, 180000);
    entityManager.persistAndFlush(voiture);
    //permet de faire persister ce tuple de l'entité Voiture
    assertThat(voiture.getId()).isNotNull();
    //permet de tester qu'un tuple de Voiture a bien été ajoutée en mémoire H2
}

@Test
public void supprimerVoiture() {
    entityManager.persistAndFlush(new Voiture ("MiolaCar","Uber","Blanche","M-2020", 2021,
    180000));
    entityManager.persistAndFlush(new Voiture ("MiniCooper","Uber","Rouge","C-2020", 2021,
    180000));
    voitureRepo.deleteAll();
    assertThat(voitureRepo.findAll()).isEmpty();
    //permet de tester si tous les tuples ont été supprimés
}
}

```

- Exécutez par Run As -> Junit Test et si tout marche bien vous allez avoir une barre verte comme ci dessous :

The screenshot shows an IDE with a Java file on the left and a test runner window on the right. The Java code defines a `VoitureRepoTest` class with two test methods: `ajouterVoiture()` and `supprimerVoiture()`. The test runner window on the right shows the test finished after 4.458 seconds, with 2/2 runs, 0 errors, and 0 failures. A green progress bar indicates successful completion. Below the progress bar, the test name `VoitureRepoTest [Runner: JUnit 5] (0.363 s)` is visible. At the bottom, there is a 'Failure Trace' section which is currently empty.

```

package org.cours;
import static org.assertj.core.api.Assertions.assertThat;
@RunWith(SpringRunner.class)
@DataJpaTest
public class VoitureRepoTest {
    @Autowired
    private TestEntityManager entityManager;
    @Autowired
    VoitureRepo voitureRepo;
    @Test
    public void ajouterVoiture() {
        Voiture voiture = new Voiture ("MiolaCar","Uber","Blanche","M-2020",
        2021, 180000);
        entityManager.persistAndFlush(voiture);
        assertThat(voiture.getId()).isNotNull();
    }
    @Test
    public void supprimerVoiture() {
        entityManager.persistAndFlush(new Voiture ("MiolaCar","Uber",
        "Blanche","M-2020", 2021, 180000));
        entityManager.persistAndFlush(new Voiture ("MiniCooper","Uber",
        "Blanche","M-2020", 2021, 180000));
        voitureRepo.deleteAll();
        assertThat(voitureRepo.findAll()).isEmpty();
    }
}

```

- Voilà!!!!